Ders 18 Genişletilmiş Notu: Fonksiyonlarla Modüler Tasarım: Kesir İşlemleri, Structure Chart, OBEB ve Min-Max Fonksiyonları

Ders 18 Genişletilmiş Notu: Fonksiyonlarla Modüler Tasarım: Kesir İşlemleri, Structure Chart, OBEB ve Min-Max Fonksiyonları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Amacı
- 2. Problem: Kesir İşlemleri
- 3. Programı Fonksiyonlara Bölmek: Tasarım Tartışması
- 4. Structure Chart
- 5. Programın Kodlanması
- 6. Kitabın Çözümü ve Stub Kavramı
- 7. Akış Diyagramında Fonksiyon
- 8. İyi Tasarım: Tekrarı Kaldırmak ve Refine ile GCD'yi Ayırmak
- 9. Öklid Algoritması
- 10. Parametrik Fonksiyonlar ve Flag Değişkeni
- 11. Dizide En Büyük ve En Küçük Eleman
- 12. Matriste En Büyük ve En Küçük Eleman: Transform and Conquer
- 13. En Yakın Nokta Çifti (Closest Pair) Problemine Giriş

1. Dersin Amacı

Rastgele sayılar konusu önceki derste bitmişti. Ortak ders programında sıradaki konu fonksiyonlardır, ama fonksiyonlar bu grupta zaten görülmüştü. Bu yüzden hoca bu dersi çeşitli örnekler üzerinden konuyu tekrar etmeye ayırdı.

Başlangıç noktası, daha önce bir öğrencinin sayısal analiz bağlamında sorduğu fonksiyon tasarımı sorusuydu. Hoca o öğrenciye tasarımını iyileştirmek için bir diyagram çizmesini önermişti. Bu derste aynı fikri, elindeki bir kitabın vaka çalışması (case study) üzerinden gösterdi. Amaç yalnızca çalışan bir program yazmak değil, programı bildiğimiz algoritma konuları ve program tasarımı mantığıyla kurmaktır.

2. Problem: Kesir İşlemleri

Uygulama, rasyonel sayılarla toplama, çıkarma, çarpma ve bölme yapan bir programdır (fraction operation). Kullanıcıdan iki kesir alınır:

$$\frac{a}{b} \quad \text{ve} \quad \frac{c}{d}$$

Yani dört tane sayı ve bir de operatör (`+`, `-`, `*`, `/`) alınır. Hoca sayıların tam sayı veya `double` olabileceğini söyledi, kodda tam sayı kullandı. Sonuç $\frac{x}{y}$ biçiminde ekrana yazılır. Bu sonuç sadeleştirilmiş (refine edilmiş) olmalıdır.

Hoca toplamanın nasıl yapılacağını bir örnekle gösterdi. Paydaları tek tek eşitlemek yerine dört değişkenle doğrudan şu formül kullanılabilir:

$$\frac{a}{b} + \frac{c}{d} = \frac{a \cdot d + b \cdot c}{b \cdot d}$$

Örnek 1: $a = 2$, $b = 3$, $c = 4$, $d = 5$ alalım.

- Pay: $2 \cdot 5 + 3 \cdot 4 = 10 + 12 = 22$
- Payda: $3 \cdot 5 = 15$
- Sonuç: $\frac{22}{15}$. Bu kesir sadeleşmez, sonuç böyle kalır.

Hoca sadeleşen bir örnek bulmak için başka sayılar da denedi:

- $\frac{2}{3} + \frac{3}{5} = \frac{2 \cdot 5 + 3 \cdot 3}{3 \cdot 5} = \frac{10+9}{15} = \frac{19}{15}$. Sadeleşmez.
- $\frac{2}{3} + \frac{2}{5} = \frac{2 \cdot 5 + 3 \cdot 2}{3 \cdot 5} = \frac{10+6}{15} = \frac{16}{15}$. Bu da sadeleşmez.

Bunun üzerine hoca, toplama işlemine bağlamadan yalnız sadeleştirme fikrini gösteren bir sonuç verdi: İşlem sonunda $\frac{22}{38}$ bulduysak bunu böyle bırakmayacağız, $\frac{11}{19}$ haline getireceğiz.

Hoca problemin düşününce çok zor görünmediğini söyledi. Asıl konu, bu programın nasıl parçalara bölüneceğidir.

3. Programı Fonksiyonlara Bölmek: Tasarım Tartışması

Hoca dersin bu bölümünü öğrencilere sırayla soru sorarak yürüttü. Sorduğu soru şuydu: Baştan sona bu problem için hangi fonksiyonları yazardınız, isimleri ne olurdu? Fonksiyonlar konusunu görmemizin nedeni, büyük bir programı küçük parçalara bölebilmektir. Burada istenen de programın parçalarının isimleridir, yani modüler bir tasarımdır.

3.1 İlk Liste

Tartışmada şu fonksiyonlar ortaya çıktı:

- Toplama, çıkarma, çarpma ve bölme için birer fonksiyon.
- Sadeleştirme fonksiyonu. Hangi işlem yapılırsa yapılsın, üretilen çıktı sonunda sadeleştirilecektir.
- Kesir alma fonksiyonu (`get fraction`). a'nın bir kısıtı yoktur, ama b sıfırdan farklı olmalıdır. Aynı durum c ve d için de geçerlidir. Aynı şeyi iki kez tekrar etmemek için tek fonksiyon yazılır.
- Ekrana yazdırma fonksiyonu. Sonuç kesir biçiminde yazılmak istenebilir.

Bir öğrenci operatör okuma için ayrı fonksiyona gerek olmadığını söyledi. Hoca bu aşamada operatör fonksiyonunu listeye almadı. Yine de ilerleyen bölümlerde `get_operator` ayrı yazıldı.

3.2 Toplama ve Çıkarmayı Birleştirmek

Hoca sordu: Toplama ile çıkarma fonksiyonunda algoritmada ne değişiyor? Yalnızca işaret. O zaman tek bir işaret farkı için iki fonksiyon yazmak yerine ikisini birleştirmek daha iyidir.

Peki birleşik fonksiyon nasıl çağrılır? Önce a, b, c, d gönderilir. Bir öğrenci işareti de (sign) göndermeyi önerdi. Sonuçlar için de iki değişken gerekir: x ve y.

Soru: x ve y'yi bu şekilde gönderirsek fonksiyonda içerikleri değişir mi?

Cevap: Hayır. İki değişken olduğu için `return` ile de döndürülemez. Adresleri gönderilmelidir, yani & kullanılır. `scanf`'i x ve y için nasıl çağırıyorsak, toplama fonksiyonunu da öyle çağırırız.

Soru: Bölmeyi de toplama fonksiyonu yapmış olmuyor mu burada?

Cevap: Hayır, bölmeye ayrıca gelinecek.

Sonra hoca daha iyi bir tasarım sordu: İşaret parametresini hiç göndermeden bu fonksiyon yazılabilir mi? Bir öğrenci toplamadan sonra sonucu -1 ile çarpmayı önerdi. Hoca toplama bittikten sonra sonucu negatif yapmanın işe yaramayacağını söyledi. Doğru cevap matematikseldir:

```
topla(a, b, -1 * c, d, &x, &y);
```

$-1 \cdot c$, c'nin negatifi demektir. Fonksiyonun içinde yine toplama yapılır, ama c'nin negatifiyle toplandığı için çıkarma yapılmış olur ve sonuç değişmez. Böylece bir parametre eksilir.

3.3 Çarpma ve Bölmeyi Birleştirmek

Aynı düşünce çarpma ve bölme için de geçerlidir:

$$\frac{a}{b} \cdot \frac{c}{d} = \frac{a \cdot c}{b \cdot d} \qquad \frac{a}{b} \div \frac{c}{d} = \frac{a \cdot d}{b \cdot c}$$

Bir öğrenci doğru çözümü buldu: Fonksiyona farklı sırayla göndermek, yani c yerine d, d yerine c:

```
carp(a, b, c, d, &x, &y);   /* carpma */
carp(a, b, d, c, &x, &y);   /* bolme  */
```

Önce $\frac{1}{c/d}$ hesaplanıp sonra çarpma da yapılabilir, ama buna gerek yoktur. Bu, fazladan bir işlemdir.

3.4 Sadeleştirme ve OBEB

Bir öğrenci sadeleştirme için şunu önerdi: x'in yarısından başlayan bir döngü kurulsun, 1'e doğru gelinsin. En kötü ihtimalle 1'e gelinir. İkisini de ortak bölen bir t sayısı bulununca ikisi de t'ye bölünür.

Hoca bunun adının ne olduğunu sordu. İkisini de bölen bir sayı arıyoruz ve ortak bölenlerin en büyüğünü istiyoruz. Bu, en büyük ortak bölen (OBEB, GCD, greatest common divisor) algoritmasıdır. Hoca öğrenciye şöyle dedi: GCD'yi bilmeden soruyu çözdün, yani Amerika'yı sen de keşfetmiş oldun.

Sadeleştirme fonksiyonunun girdi ve çıktısı da konuşuldu. x ve y alınacak, sadeleşecek ve geri gelecektir. Yeni değerler bize gelsin diye adresleri gönderilir.

Yarısından başlama fikrinin neden yetersiz olduğu kod yazılırken ortaya çıktı (Bölüm 5.7).

3.5 Kesir Alma Fonksiyonu Nasıl Çağrılır?

Hoca kesir alma fonksiyonuna `GF` dedi. Bu fonksiyon önce a ve b, sonra c ve d için çağrılacaktır. Sayılara genel olarak `S1` ve `S2` diyelim. Fonksiyon genelde `main`'den çağrılır. `S1` ve `S2`'nin içine girilen değerler kaydedilirse kullanıcıdan veri almış oluruz. Ama `GF(S1, S2)` biçiminde içerikleri değişmez. Değişmesi için başlarına `&` yazılır. Bu, call by reference mantığıdır.

Fonksiyonun içinde ise bir `scanf` vardır ve burada sık bir karışıklık yaşanır. `S1` fonksiyona zaten pointer olarak geldiği için `scanf`'te `&S1` yazılamaz:

```
scanf("%d", S1);   /* dogru: S1 zaten pointer */
```

`&S1` yazılırsa pointer'ın pointer'ı olur.

3.6 Ekrana Yazdırma Fonksiyonu Nasıl Çağrılır?

Yazdırma fonksiyonu bir şey döndürmez, içinde yalnız `printf` vardır, bu yüzden `void`'dir. Asıl soru, hangi parametrelerin nasıl gönderileceğidir. Örneğin ekranda $\frac{3}{8} + \frac{7}{17} = \ldots$ görmek istiyoruz. Fonksiyon her çağrıldığında bir kesir yazacaktır.

Hoca bu örneğin sonucunu hesaplamadı. Hesaplanırsa (ara adım eklendi):

$$\frac{3}{8} + \frac{7}{17} = \frac{3 \cdot 17 + 8 \cdot 7}{8 \cdot 17} = \frac{51 + 56}{136} = \frac{107}{136}$$

Burada bir öğrenci adresle göndermeyi önerdi. Hoca şöyle açıkladı: `&` ile göndermek, içerik orada değişirse bu değişikliğin dışarıya yansıması demektir. Ekrana yazdırırken içeriği değiştirmeye gerek yoktur. Gerek yoksa adres yazmaya da gerek yoktur. `a` ve `b` fonksiyona değerle gider, bir değişikliğe uğramaz.

Hoca başka bir seçenek de söyledi: Tek çağrıda hepsini yazdırmak için a, b, c, d, x, y ve operatör birlikte gönderilebilir. Bu da olabilir.

4. Structure Chart

Hoca, oluşturulan listenin fikir verdiğini ama biraz karmaşık olduğunu söyledi. Sadeleştirme dedik, altından OBEB çıktı, oradan başka yere geçtik. Bunu sistematik hale getiren araç structure chart'tır (yapısal diyagram).

Slaytta gösterilen diyagramın yapısı şöyledir. Parantez içindeki oklar parametre akışını gösterir:

```
main / test / driver
|
|-- Get fraction problem       (çıkış: pay1, payda1, operatör, pay2, payda2)
|     |-- kesir al             (çıkış: pay, payda; iki kez çağrılır)
|     `-- get operator         (çıkış: operatör)
|
|-- Continue?                  (devam etmek ister misiniz)
|
|-- Compute result             (giriş: iki kesir ve operatör; çıkış: x, y)
|     |-- toplama / çıkarma
|     |-- çarpma / bölme
|     `-- Reduce fraction      (x ve y'yi sadeleştirir)
|           `-- GCD            (giriş: iki sayı; çıkış: OBEB)
|
`-- Print problem and result   (giriş: birinci kesir, operatör, ikinci kesir, sonuç; çıkış yok
```

Hoca diyagramı yukarıdan aşağıya okudu:

- **Üst modül:** `main` olmak zorunda değildir. `main`'den `test` adında bir fonksiyona gidip yapılacak işlemleri oradan çağırabilirsiniz. Buna test veya driver da denebilir.
- **Get fraction problem:** Fonksiyon bize pay, payda, operatör, pay ve payda değerlerini geri verir. Kendi içinde bir kesir alma, bir de operatör alma fonksiyonu çağırır. Kesir alma ayrı yazılmıştır, çünkü aynı işlem iki kesir için tekrarlanır. Üç dört satırlık olsa da kod tekrarı (duplicated code) olmasın, yönetilebilir olsun diye ayrılır. Pay ve payda `&` ile gönderilir. Operatör için ayrı fonksiyon şart olmayabilir, ama `printf` ile mesaj yazıp `scanf` ile almak ve dört işlem dışındaki girişleri hata saymak için ayrı bir fonksiyon iyi olur.
- **Continue?:** Hesaplar bittikten sonra kullanıcı yeniden işlem yapmak isteyebilir. Bu bir seçenektir, her zaman yazmak gerekmez.
- **Compute result:** `switch-case` veya `if` ile de yazılabilir, burada ayrı fonksiyon yapılmıştır. Girdisi kesirlerin değerleri ve operatör, çıktısı x ve y'dir. İçinde toplama-çıkarma ile çarpma-bölme ayrı bölümlerdir ve bu aşamada sadeleştirme yoktur. Sonuçlar dışarı verilmeden önce `Reduce fraction` işleminden geçer. Bu işlem iki sayıyı GCD fonksiyonuna gönderir, gelen OBEB ile sadeleştirir, sadeleşmiş pay ve paydayı döndürür.
- **Print problem and result:** Birinci kesir, operatör, ikinci kesir ve sonuç gelir. Geri dönüş yoktur, çünkü yalnız ekrana yazdırılır.

Structure chart, programdaki fonksiyonların neler olacağını ve hangi parametreleri alacağını gösterir. Hoca, kendi projelerinizde benzerini çizmenin kağıt üzerinde daha iyi fikir vereceğini söyledi. "Bu fonksiyonu mu yazsam, yoksa iki fonksiyonu birleştirsem mi?" sorusunun cevabı orada görülür. Az önce toplama ile çıkarmayı ve çarpma ile bölmeyi algoritmik olarak aynı oldukları için birleştirdik.

> **Hocanın vurgusu:** Structure chart, kodlamaya başlamadan önce fonksiyonları ve parametrelerini sistematik olarak görmeyi sağlar. Kendi projelerinizde de çizin.

5. Programın Kodlanması

Hoca diyagramı birebir izlemedi, çünkü o biçimde kod uzuyordu. Kesir alma fonksiyonunu doğrudan `main`'den çağırdı. Değişkenler tam sayı olarak tanımlandı: `a`, `b`, `c`, `d` girdi, `x`, `y` sonuç.

5.1 `get_fraction`

`main`'den `a` ve `b`'nin adresleri gönderilir, tıpkı `scanf`'te olduğu gibi. Sonra aynı çağrı `c` ve `d` için yapılır:

```
get_fraction(&a, &b);
get_fraction(&c, &d);
```

Fonksiyon bir şey döndürmediği için `void`'dir. Adresle çağrıldığı için parametrelerin türü pointer olmak zorundadır. Hoca isimleri de açıklayıcı olsun diye pay ve payda yaptı (nominator, denominator).

Pay için `scanf`'in nasıl yazılacağını bir öğrenciye sordu. Cevap: `scanf("%d", pay);`. Öğrenci "adresini zaten aldığımız için gerek kalmıyor" dedi, hoca da şöyle düzeltti:

> **Hocanın vurgusu:** "Gerek kalmıyor" değil, zorunlu. Burada `&pay` yazarsak adresin adresi gibi bir şey olur, pay değişkeninin adresine yazmaya çalışırız. İkinci dereceden pointer'a gider.

Payda sıfır olamaz. Bunu `if` ile kontrol edip tekrar istemek yetmez, çünkü `if` ile döngü sağlanamaz. `do-while` daha uygundur, çünkü giriş en az bir kez yapılacaktır. Koşul, "payda sıfıra eşitse bir daha iste" olur:

```
void get_fraction(int *pay, int *payda) {
    printf("Pay giriniz: ");
    scanf("%d", pay);
    do {
        printf("Payda giriniz: ");
        scanf("%d", payda);
    } while (*payda == 0);
}
```

Koşulda `payda` yazılamaz, `*payda` yazılır. Çünkü adrese değil, adresin içindeki değere bakılır.

5.2 `get_operator`

Hoca önce bu fonksiyonun da `void` olup operatörü adresle almasını düşündü, sonra vazgeçti. Tek bir değer döneceği için `char` döndürmek daha iyidir, en azından pointer ile uğraşılmaz:

```
op = get_operator();
```

Geçersiz girişi engellemek için yine `do-while` kullanıldı ve okuma şöyle yazıldı:

```
scanf(" %c", &c);
```

`%c`'den önceki boşluk önemlidir. Daha önce pay ve payda girilip Enter'a basıldı. Bu Enter bufferda saklı kalır ve `%c` kullanıcının gireceği karakter yerine onu okuyabilir. Araya bir boşluk koymak, asıl karakterin okunmasını sağlar. Hoca bu farkın test ederken görüleceğini söyledi. Nitekim ilerleyen bir denemede boşluk yazılmadığı için "Operatör giriniz" mesajı iki kez çıktı (Bölüm 6.3).

Koşulu yazarken bir öğrenci önce `c == '+'` dedi. Hoca bunun "artı ise tekrar yap" anlamına geldiğini gösterdi. Doğrusu "dört operatörden biri değilse tekrar al"dır: artı değilse **ve** eksi değilse **ve** çarpı değilse **ve** bölü değilse.

```
while (c != '+' && c != '-' && c != '*' && c != '/');
```

5.3 `print_fraction` ve Yazdırma

Fonksiyon iki tam sayıyı değerle alır ve kesri yazar. Matematikteki gibi bir kesir çizgisi çizemeyiz, araya normal `/` işareti konur:

```
void print_fraction(int pay, int payda) {
    printf("%d/%d", pay, payda);
}
```

`main`'de önce birinci kesir, sonra operatör (`%c` ile, önünde ve arkasında birer boşlukla), sonra ikinci kesir yazdırılır. Hoca, her şeyi tek `printf` ile de yazdırabileceğimizi ama böylesini daha doğru bulduğunu söyledi. Çünkü program daha modüler hale gelir.

5.4 İlk Derleme ve Prototipler

İlk derlemede derleyici `get_fraction` ve diğer fonksiyonlar için ikaz verdi. Hoca prototipleri ekledi ve şunu söyledi: Özellikle pointer kullanıyorsak prototipleri yazmak durumundayız.

> **Ders sonrası not:** Prototip ihtiyacı büyük olasılıkla pointer'a özgü değildir. Bir fonksiyon, çağrıldığı yerden sonra tanımlanıyorsa çağrıdan önce bildirilmesi gerekir. Bu dersteki kodda da bu yüzden gereklidir.

Prototiplerde değişken isimleri silinebilir, ama tipler kalmalıdır. Dizi ve matris de birer veri tipi olarak korunur. Matriste sütun bilgisi yazılır:

```
void get_fraction(int *, int *);
int  fonk1(int []);            /* dizi   */
int  fonk2(int [][50]);        /* matris */
```

5.5 İlk Çalıştırma ve && Kontrolü

İlk denemede şu girişler yapıldı: Pay 3, payda 0, program paydayı tekrar istedi, payda 4. İkinci kesir 4 ve 6. Operatör olarak önce mod işareti girildi. Program kabul etmedi ve beklemeye geçti. Sonra geçerli bir operatörle devam edildi ve ekranda şu görüldü: `3/4 + 4/6`.

> **Ders sonrası not:** Hoca bu sırada "sonsuz döngüye girdi" dedi. Ekranda büyük olasılıkla sonsuz döngü değil, yeni giriş bekleyen `scanf` vardı. Hocanın da belirttiği gibi "Operatör giriniz" mesajı o anda `do-while`'ın dışındaydı, bu yüzden tekrar yazılmıyordu ve program takılmış gibi görünüyordu. Ders sonrası denemede 3, 0, 4, 4, 6, `%`, `+` girişleri 3/4 + 4/6 = 17/12 ile sona erdi.

Aradan sonra hoca, koşulun doğruluğunu bir daha kontrol etti. "Veya mı, ve mi" diye durdu ve değerler üzerinden izledi:

Giriş	`c != '+'`	`c != '-'`	`c != '*'`	`c != '/'`	&& sonucu	Davranış
`*`	doğru	doğru	yanlış	doğru	yanlış	döngüden çıkılır
`%`	doğru	doğru	doğru	doğru	doğru	tekrar sorulur

Yani && doğrudur. Hoca aynı anda 2/3 ve 4/5 ile `+` işlemini de denedi, ekranda `2/3 + 4/5` göründü. Bu, bir öğrencinin "artı yerine bölü yazdırdı" gözleminin üzerine yapılmış bir kontroldü, hoca sorun görmedi.

> **Ek bilgi (derste anlatılmadı):** "Dört operatörden biri ise çık" ifadesinin değili, büyük olasılıkla De Morgan kuralı olarak bilinen kurala göre "hepsinden farklıysa devam et" olur. Bu nedenle koşulda && gerekir. `||` yazılırsa koşul her karakter için doğru olur, çünkü bir karakter aynı anda dört operatöre de eşit olamaz. Bu durumda döngüden hiç çıkılamaz (python ile beş girdi için doğrulandı).

5.6 `SUM` ve `MUL`

Toplama fonksiyonuna + adı verilemez. + bir fonksiyon adı olamaz, değişken adlarındaki kural burada da geçerlidir. Hoca `sum` adını seçti. Çarpma için de `multiplication` düşündü. Kısa notla uyumlu olarak burada `SUM` ve `MUL` yazıyoruz.

Sayılar tam sayı alındı, istenirse `float` da olabilirdi. Sonuç değişkenleri fonksiyonda değişeceği için pointer olarak alınır. `*x`, "x'in gösterdiği yerdeki değer" demektir:

```
void SUM(int a, int b, int c, int d, int *x, int *y) {
    *x = a * d + b * c;
    *y = b * d;
    refine(x, y);
}

void MUL(int a, int b, int c, int d, int *x, int *y) {
    *x = a * c;
    *y = b * d;
    refine(x, y);
}
```

`x` ve `y` burada zaten pointer olduğu için `refine`'a `&` olmadan verilir. `refine` içeriği değiştirir, sadeleşmiş değerler `SUM` veya `MUL`'ün çağrıldığı yere yansır.

5.7 `refine` ve `gcd`

Sadeleştirmek için önce OBEB bulunur, sonra iki sayı OBEB'e bölünür. OBEB için ayrı bir fonksiyon yazılır. Ona x ve y'nin sayısal değerleri gönderilir, yani `*x` ve `*y`:

```
void refine(int *x, int *y) {
    int g = gcd(*x, *y);
    *x = *x / g;
    *y = *y / g;
}
```

`gcd` fonksiyonuna değer gönderildiği için değerle alınır. Algoritma şöyledir:

1. İki sayının en küçüğünü bul. Pay mı payda mı küçük, bilmiyoruz.
2. Bu değerden başlayarak geriye doğru git.
3. Her değerde, sayının ikisini de bölüp bölmediğine bak.
4. İkisinden birini bile bölmüyorsa değeri bir azalt.
5. İkisini de bölen ilk değer OBEB'dir, onu döndür.

Hoca en küçüğü önce `if-else` ile yazmaya başladı, sonra koşul operatörünü (conditional operator) hatırlattı: Koşul doğruysa `x`, değilse `y` atanır. Eşitlik de bu ifadeyle karşılanmış olur.

Bölüm 3.4'teki "yarısından başla" fikri burada düzeltildi. $\frac{11}{22}$ örneğinde OBEB 11'dir, yani küçük sayının kendisidir. 11'in yarısından başlarsak onu kaçırırız. Bu yüzden arama küçük sayının kendisinden başlar:

```
int gcd(int x, int y) {
    int min = x < y ? x : y;
    while (x % min != 0 || y % min != 0)
        min--;
    return min;
}
```

Hoca önce "ve" diye başladı, sonra koşulun "veya" olması gerektiğini söyledi: `x`'i veya `y`'yi bölemiyorsa `min` azaltılır.

Soru: Bu `while`'da başka bir koşul gerekmez mi? Döngü sonsuza gitmez mi?

Cevap: `min` bir yerde 1 olacaktır. 1 her sayıyı böler, iki koşul da yanlış olur ve döngüden çıkılır. Sıfıra gelinmez.

5.8 İlk Test: $\frac{2}{3} + \frac{2}{3}$

Hoca önce toplama çağrısını `main`'e eklemeyi unuttu, ekranda sonuç çıkmadı. Çağrıyı ekledi:

```
SUM(a, b, c, d, &x, &y);
```

Giriş 2, 3, 2, 3 ve + iken ekranda $\frac{4}{3}$ göründü. Hoca sayıları önce yanlış çarpıp $\frac{12}{6}$ dedi, ama sonra düzeltti. Adım adım:

- Pay: $2 \cdot 3 + 3 \cdot 2 = 6 + 6 = 12$
- Payda: $3 \cdot 3 = 9$
- OBEB: $\gcd(12, 9)$ için min $= 9$ ile başlanır. 9, 12'yi bölmez. 8, 7, 6, 5, 4 de ikisini birden bölmez (ara adım eklendi). 3 ikisini de böler, OBEB 3'tür.
- Sonuç: $\frac{12/3}{9/3} = \frac{4}{3}$

  Ders sonrası not: Hoca bir an "OBEB'imiz 9" dedi. 12 ile 9'un OBEB'i 3'tür. Program da $\frac{4}{3}$ vererek bunu doğrular (python ve gcc ile doğrulandı).

5.9 Negatif Sayılar

Soru: Bu negatif sayılarda çalışır mı? Minimumu azaltıyoruz ya, çalışmaz gibi geldi.

Cevap: O zaman sayıların mutlak değerini almak gerekir. Sıfırdan küçükse pozitife dönüştür, iki sayı için de bunu yap, sonra algoritmayı çalıştır. Hoca bunu iyice genelleştirmek gerektiğini söyledi, ama kodda yazmadı.

5.10 `do_operation`

`main`'de işlem seçimini tek bir çağrıyla toplamak için `do_operation` yazıldı: Girdisi `a`, `b`, `c`, `d`, operatör; çıktısı `&x`, `&y`. Parametre listesi uzundur. Hoca, ikinci sınıfta görülecek fonksiyon pointer'larının bu işi daha kısa yazdırdığını ve nesneye yönelik kavramlarda da böyle bir şey olduğunu söyledi.

Hoca önce `+` veya `-` ise aynı çağrıyı yapmayı düşündü, sonra operatörlerin ayrı `if`'lerle ele alınması gerektiğini söyledi. Çünkü her biri farklı parametreyle çağrılır. Buradaki `x` ve `y` artık pointer'dır, doğrudan gönderilir. Son dal için kontrol gerekmez, çünkü `get_operator` başka bir karakter döndürmez:

```
void do_operation(int a, int b, int c, int d, char op, int *x, int *y) {
    if (op == '+')
        SUM(a, b, c, d, x, y);
    else if (op == '-')
        SUM(a, b, -c, d, x, y);
    else if (op == '*')
        MUL(a, b, c, d, x, y);
    else
        MUL(a, b, d, c, x, y);
}
```

Hoca `-1 * c` yazılabileceğini, ama C'de eksi operatörünün sayıyı doğrudan negatife çevirdiğini söyledi. Yani `-c` de yazılabilir.

Deneme: $\frac{2}{3} + \frac{2}{3} = \frac{4}{3}$. Sonra bölme denendi: $\frac{2}{3} \div \frac{2}{3}$ ekranda `1/1` verdi. Adım adım (ara adım eklendi): `MUL(2, 3, 3, 2, ...)` çağrılır, $x = 2 \cdot 3 = 6$, $y = 3 \cdot 2 = 6$, $\gcd(6, 6) = 6$, sonuç $\frac{1}{1}$.

Bu işlemleri baştan yapmak istersek `main`'e bir `do-while` döngüsü konur. Hoca bunun bir sürü ayrıntısı olduğunu, ama burada amacın fonksiyonları etkili kullanabilmek olduğunu söyledi.

6. Kitabın Çözümü ve Stub Kavramı

6.1 Kitaptaki `main` ve `get_operator`

Hoca sonra kitabın koduna geçti. `main`'in dönüş tipi üst satıra yazılabilir, bu hata değildir. Kitabın `main`'inde önce değerler alınır, sonra operatör alınır. İşlem seçimi ayrı bir fonksiyonda

değil, doğrudan `main`'dedir: Toplamaysa toplama, değilse çarpma çağrılır. Sonra `reduce fraction` çağrılır. Bizim kodumuzdan farkı budur. Ardından kesirler, operatör ve eşittir yazılır, kullanıcıya devam etmek isteyip istemediği sorulur.

Kitabın `get_operator`'ı daha ayrıntılıdır. `scanf` ile bir operatör alınır. Bu dört işlemden biri değilse hata mesajı verilip tekrar istenir. Enter da bir karakter olarak işlendiği için fonksiyon uzamıştır. Hocaya göre bu gereksiz bir ayrıntıdır.

Soru: Bu başka bir dil mi, C programı mı?

Cevap: C programıdır. Bu işi `for` ile de yapabilir. `for` döngüsünde başlangıç ve artırma kısımlarını yazmak zorunda değiliz, oralar boş kalabilir. Önemli olan şarttır, burada yalnız şart yazılmıştır.

6.2 Kitapta Negatif Sayılar ve `find_gcd` Stub'ı

Kitabın toplama fonksiyonu negatif durumları da ele alır: Pay ve paydanın çarpımına bakılır. Biri negatifse ikisinin de işareti değiştirilir, sonra sadeleştirmeye gidilir.

> **Ders sonrası not:** Hoca çarpım için "sıfırdan farklıysa, yani bir tanesi negatifse" dedi. "Biri negatif" olması çarpımın sıfırdan **küçük** olmasıdır. Kitaptaki koşul büyük olasılıkla budur.

Hocanın asıl göstermek istediği şey `find_gcd` fonksiyonudur. İçine bakınca bir `printf` ve bir `scanf` görülür. Fonksiyon sayıları, örneğin 8 ve 27'yi ekrana yazar ve OBEB'i kullanıcıdan ister. Yani OBEB hesaplamaz. Biraz anlamsız görünebilir, ama bu bir tasarım konusudur.

Buna **stub** denir. Hoca şöyle açıkladı: Bir ekip düşünün. OBEB fonksiyonunu başkası yazacak, siz de sadeleştirme fonksiyonunuzu test etmek istiyorsunuz. Bunun için GCD'ye ihtiyacınız var, ama o henüz hazır değil. Bu arada fonksiyonu işinizi görecek şekilde geçici olarak doldurursunuz. Bu gerçek bir algoritma değildir, tek görevi diğer modüllerin test edilmesini sağlamaktır.

6.3 Enter Sorunu Ekranda

Bir öğrencinin isteğiyle program tekrar çalıştırıldı: 2/3 ve 2/3 ile toplama.

Soru: Ekranda iki kere "Operatör giriniz" yazıyor. Hata mı var, biz mi iki kere yazdırıyoruz?

Cevap: Mesaj bir kez yazılıyor, değer alınmıyor, bir kez daha yazılıyor. Bunun nedeni `%c`'den önceki boşluğun bırakılmamış olmasıdır. Önceki girişten kalan Enter, operatör olarak okunur. Dört işlemden biri olmadığı için döngü tekrar eder.

Ders sonrası denemede boşluk kaldırıldığında mesajın iki kez çıktığı doğrulandı (gcc).

7. Akış Diyagramında Fonksiyon

Hoca, fonksiyonların ne olduğu kısmını geçip diğer dersin slaytındaki akış diyagramına geldi. Önce C'den bağımsız düşünelim: Bir `F` fonksiyonu x, y, z gibi parametreler alır ve algoritmik olarak birden fazla değer döndürebilir. Fonksiyonun yaptığı şey değer almak ve değer döndürmektir, aradaki algoritmayı siz yazarsınız.

Burada `return` ifadesinden söz edilmektedir:

- C, Java, C#, C++: `return` ile 0 veya 1 değer döner. 0 ise `void`'dir, değilse `int` veya kendi tasarladığınız bir tip olabilir. İki değer gerekiyorsa call by reference kullanılmak zorundadır.
- MATLAB, Python: Birden fazla değer döndürmek mümkündür.
- Akış diyagramı: Birden fazla değer gösterilebilir.

Soru: İki değer döndürmek derken neyi kastediyorsunuz? Örnek verebilir misiniz?

Cevap: Bir dizinin en büyük elemanını bulacağız. Hem elemanın değerini, hem indisini istiyoruz.

Akış diyagramında fonksiyon şöyle gösterilir:

```
Fonksiyon başlığı (sol tarafa bakan ok):

    üstüne:  MaxBul(A, N)         fonksiyon adı ve giriş parametreleri
    <--------------------
    altına:  maxD, maxIndis       çıkış değerleri

Gövde:
  1. Başlangıçta en büyük olarak ilk eleman kabul edilir.
  2. Diğer elemanlar tek tek bununla karşılaştırılır.
  3. Daha büyük bulunursa en büyük değer ve indisi güncellenir.
  4. Elemanlar bitince maxD ve maxIndis döner.
  5. stop

Çağrı:
  MaxBul(A, N) çağrılır; dönen iki değer sırasıyla iki değişkene alınır:
  önce maxD, sonra maxIndis.
```

Tipler yazılmaz, ama yazılabilir. Fonksiyonun bitişine `stop` yazılır. Buradaki dönüş C'deki `return` değildir, C'de böyle bir şey yoktur. Akış diyagramı algoritmayı daha genel düşünmek içindir. MATLAB ve Python'da da kullanım bu şekildedir.

Aynı slaytta matrisi kullanıcıdan alan bir fonksiyon da vardı. Hoca bunu bilinen bir konu sayarak kısaca geçti: Tanımlamada sütun bilgisi belirtilmek zorundadır, satır ve sütun sayısı ayrıca okunur, matris okunur, ilk ve son eleman ekrana yazılır.

8. İyi Tasarım: Tekrarı Kaldırmak ve Refine ile GCD'yi Ayırmak

8.1 Tasarım Değişikliğinin Maliyeti

Fonksiyonlar parametrik yazılmalı, değişebilecek şeyler parametre olarak alınmalıdır. Bu, fonksiyon sayısını azaltabilir veya duruma göre artırabilir. Yalnız "azalır" demek işi kısıtlamak olur.

> **Hocanın vurgusu:** Tasarımında bir değişiklik yapacaksan ve bu değişiklik için bütün programı baştan aşağıya yenilemen gerekiyorsa, o kötü bir tasarımdır.

Hoca bunu kendi kodunda gösterdi. `refine`, hem `SUM` hem `MUL` tarafından çağrılıyordu. Bu bir kod tekrarıdır. Kitaptaki gibi yapmak için bu çağrıları sildi ve `do_operation`'ın sonuna tek bir çağrı koydu. Hangi işlem yapılırsa yapılsın, sonuç mutlaka sadeleşmiş olur:

```c
void do_operation(int a, int b, int c, int d, char op, int *x, int *y) {
    if (op == '+')
        SUM(a, b, c, d, x, y);
    else if (op == '-')
        SUM(a, b, -c, d, x, y);
    else if (op == '*')
        MUL(a, b, c, d, x, y);
    else
        MUL(a, b, d, c, x, y);
    refine(x, y);
}
```

Hem tekrar kalktı, hem de yapı bozulmadı. `refine`'ın içinde hiçbir değişiklik gerekmedi.

8.2 Programın Tamamı

Dersteki son haliyle program aşağıdadır. Ders sonrasında `gcc -Wall` ile uyarısız derlendi ve hocanın örnekleriyle çalıştırıldı.

```c
#include <stdio.h>

void get_fraction(int *, int *);
char get_operator();
```

```c
void print_fraction(int, int);
void SUM(int, int, int, int, int *, int *);
void MUL(int, int, int, int, int *, int *);
void refine(int *, int *);
int gcd(int, int);
void do_operation(int, int, int, int, char, int *, int *);

int main() {
    int a, b, c, d, x, y;
    char op;

    get_fraction(&a, &b);
    get_fraction(&c, &d);
    op = get_operator();

    do_operation(a, b, c, d, op, &x, &y);

    print_fraction(a, b);
    printf(" %c ", op);
    print_fraction(c, d);
    printf(" = ");
    print_fraction(x, y);
    printf("\n");
    return 0;
}

void get_fraction(int *pay, int *payda) {
    printf("Pay giriniz: ");
    scanf("%d", pay);
    do {
        printf("Payda giriniz: ");
        scanf("%d", payda);
    } while (*payda == 0);
}

char get_operator() {
    char c;
    do {
        printf("Operator giriniz (+ - * /): ");
        scanf(" %c", &c);
    } while (c != '+' && c != '-' && c != '*' && c != '/');
    return c;
}

void print_fraction(int pay, int payda) {
    printf("%d/%d", pay, payda);
}

void SUM(int a, int b, int c, int d, int *x, int *y) {
    *x = a * d + b * c;
    *y = b * d;
}

void MUL(int a, int b, int c, int d, int *x, int *y) {
    *x = a * c;
    *y = b * d;
}

void refine(int *x, int *y) {
    int g = gcd(*x, *y);
```

```
    *x = *x / g;
    *y = *y / g;
}

int gcd(int x, int y) {
    int min = x < y ? x : y;
    while (x % min != 0 || y % min != 0)
        min--;
    return min;
}

void do_operation(int a, int b, int c, int d, char op, int *x, int *y) {
    if (op == '+')
        SUM(a, b, c, d, x, y);
    else if (op == '-')
        SUM(a, b, -c, d, x, y);
    else if (op == '*')
        MUL(a, b, c, d, x, y);
    else
        MUL(a, b, d, c, x, y);
    refine(x, y);
}
```

Hocanın derste kullandığı girişlerin ders sonrası sonuçları:

Giriş	Program çıktısı
2/3 + 2/3	4/3
2/3 / 2/3	1/1
2/3 + 4/5	22/15
2/3 - 2/3	Floating point exception

8.3 Çıkarmada Çıkan Hata

Hoca bu değişiklikten sonra çıkarmayı denedi ve program hata verdi. Hata hakkında "0 bölü 0'dan dolayı" dedi ve bunun yazılmamış, ayrı bir OBEB sorunu olduğunu, düzeltilmesi gerektiğini belirtti.

> **Ders sonrası not:** Hoca "0 bölü 0" dedi. Girişin 2/3 ve 2/3 olduğu varsayılırsa pay $2 \cdot 3 + 3 \cdot (-2) = 0$, payda $3 \cdot 3 = 9$ olur, yani kesir 0/9'dur. Sorun `gcd(0, 9)` çağrısındadır: `min` 0 olur ve `x % 0` hesaplanır. Sıfıra bölme programı kapatır. Ders sonrası denemede program `Floating point exception` ile kapandı. Bölüm 5.7'deki "1'e gelince durur" açıklaması `min` en baştan 0 değilse geçerlidir.

> **Ders sonrası not:** Hocanın söylediği mutlak değer ve bu sıfır durumu `refine` içinde birlikte çözülebilir. Negatif sayıda mutlak değer alınmazsa `min` negatif başlar ve azaldıkça sıfırdan uzaklaşır, 1'e hiç gelmez. Aşağıdaki sürüm ders sonrasında `gcc -Wall` ile denendi: 0/9 için 0/1, 12/9 için 4/3 verdi. Ayrıca payda negatifse işaret büyük olasılıkla paya taşınmalıdır, kitabın işaret düzenlemesi bu amaçla yapılmış olabilir.

```
void refine(int *x, int *y) {
    int g;
    if (*x == 0) {       /* derste yazilmadi */
        *y = 1;
        return;
    }
    g = gcd(*x < 0 ? -*x : *x, *y < 0 ? -*y : *y);
    *x = *x / g;
    *y = *y / g;
}
```

Ders sonrası not: `get_fraction` yalnız paydanın sıfır olmasını engeller. Bölmede `MUL(a, b, d, c, ...)` çağrıldığı için ikinci kesrin payı 0 ise sonucun paydası $b \cdot 0 = 0$ olur. Bu yüzden bölme seçildiğinde $c \neq 0$ da büyük olasılıkla ayrıca kontrol edilmelidir. Derste bu durum konuşulmadı.

8.4 Refine ile GCD Neden Ayrı?

Hoca sordu: OBEB'i `refine`'ın içine de yazabilirdim, neden ayırdım? Bir öğrenci "GCD'yi değiştirmek isteyebiliriz" dedi. Hoca bunu iyi bir yorum olarak nitelendirdi ve bir neden daha ekledi: GCD başka bir yerde de lazım olabilir. Program genişlediğinde GCD yalnız `refine` için kalmaz. Zaten OBEB sadece kesir sadeleştirmede kullanılmaz, OBEB-OKEK problemleri diye bir konu bile vardır.

Bu, hocanın daha önce söylediği **low coupling, high cohesion** ilkesidir. GCD `refine`'ın içine eklenirse bağlılık artar. O fonksiyonda hem sadeleştirme, hem GCD hesabı yapılır. Oysa fonksiyon adına uygun çalışmalıdır, `refine` yalnız sadeleştirmeyi bilmelidir. Hocanın benzetmesiyle: OBEB hesabı bu modülün işi değildir, siz gidip oradan servis alırsınız. Stub'ı anlatırken kastedilen durum da budur.

Ek bilgi (derste anlatılmadı): Coupling, modüller arasındaki bağımlılığın derecesini; cohesion, bir modülün içindekilerin tek bir işe ne kadar hizmet ettiğini büyük olasılıkla anlatan yazılım tasarımı terimleridir. İstenen durum, modüller arası bağımlılığın az (low coupling), modül içi bütünlüğün yüksek (high cohesion) olmasıdır.

9. Öklid Algoritması

Dersteki `gcd` iteratiftir, küçük sayıdan geriye doğru dener. Hoca daha önce anlattığı Öklid algoritmasını hatırlattı. Önce 18 ile 4 dedi, sonra örneği 18 ile 10 olarak değiştirdi.

Mevcut algoritma ile $\gcd(18, 10)$**:** 10'dan geriye doğru gidilir. 10 ikisini de böler mi? Hayır. 9? Hayır. Bu şekilde 2'ye kadar gelinir.

Öklid ile $\gcd(18, 10)$**:** İkinci sayı birinci yere, iki sayının bölümünden kalan ikinci yere yazılır. İkinci sayı 0 olana kadar devam edilir:

Adım	Çağrı	Kalan	Sonraki çağrı
1	$\gcd(18, 10)$	$18 \bmod 10 = 8$	$\gcd(10, 8)$
2	$\gcd(10, 8)$	$10 \bmod 8 = 2$	$\gcd(8, 2)$
3	$\gcd(8, 2)$	$8 \bmod 2 = 0$	$\gcd(2, 0)$
4	$\gcd(2, 0)$	ikinci sayı 0	OBEB = 2

Hocanın sayımıyla diğer yöntem yaklaşık 10 denemede, Öklid ise 4 adımda sonuca ulaştı.

Ders sonrası not: Hoca geriye doğru deneme için "10 defa diyelim" dedi. Tam sayı 9 denemedir: 10, 9, 8, 7, 6, 5, 4, 3, 2 (python ile doğrulandı).

Hocanın bu noktadaki asıl mesajı tasarımla ilgilidir: Algoritmayı iyileştirmek istersem yalnızca `gcd` fonksiyonunu değiştirmem yeterlidir. `gcd`'nin işi OBEB hesaplamaktır, kesir sadeleştirmek değil. Hoca kodun çok basit olduğunu, gerekirse sonra yazılacağını söyledi.

Ek bilgi (derste anlatılmadı): Öklid algoritması aşağıdaki gibi yazılabilir. Yalnız bu fonksiyon değiştirilir, geri kalan program aynen kalır. Ders sonrası denemede `gcc -Wall` ile $\gcd(18, 10) = 2$ ve $\gcd(12, 9) = 3$ çıktı. $\gcd(0, 9)$ de 9 verdi, yani bu sürüm Bölüm 8.3'teki sıfıra bölme hatasına düşmez.

```c
int gcd(int x, int y) {
    int r;
    while (y != 0) {
        r = x % y;
        x = y;
        y = r;
    }
```

```
        return x;
    }
```

10. Parametrik Fonksiyonlar ve Flag Değişkeni

Dört elemanlı bir diziyi okuyan fonksiyon yerine n elemanlı diziyi okuyan fonksiyon yazmak daha önemlidir. Parametrik yazmaktan kasıt budur. Modülleri ayırmak da gerekir.

Küçükten büyüğe ve büyükten küçüğe sıralama ya da diziyi düz veya ters yazdırma gibi seçenekler için aynı kodu tekrar tekrar yazmak gerekmez. Bir flag değişkeni ile tek fonksiyonda toplanabilir. Örneğin sıralama algoritması bellidir, iki yön arasındaki fark yalnızca karşılaştırmadadır.

11. Dizide En Büyük ve En Küçük Eleman

Hoca kodun başlangıç kısmını hazır kopyaladı ve fonksiyonu yazdı. Beş elemanlı, ilk değer verilmiş bir dizi vardır. Bir fonksiyon hem en büyüğü, hem en küçüğü bulacak; bu fonksiyon sonra matris için de kullanılacaktır.

Fonksiyonun parametreleri şunlardır: dizi, eleman sayısı, operasyon seçimi ve indisin saklanacağı adres. Değer `return` ile, indis call by reference ile döner.

Hoca aslında indisi bildiğimizde değere de ulaşabileceğimizi, değeri ayrıca döndürmeye gerek olmadığını söyledi. Ama akış diyagramındaki algoritma C'de istense ve `return` kullanımı zorunlu kılınsa ne yapardık? İki yol vardır: İndis `return` ile, değer parametreyle ya da tam tersi. Burada ikincisi seçildi.

> **Ders sonrası not:** Hoca seçimi anlatırken bir an "1 gönderirsem minimumu bulsun" dedi. Yazdığı kodda `op == 1` en büyüğü bulur, başka değer en küçüğü bulur. Matris fonksiyonunda da benzer bir karışıklık var, doğrusu koddaki gibidir.

Sonuç olarak dizinin sıfırıncı elemanı kabul edilir, en büyük de arasak en küçük de arasak öyledir. Döngü 0'dan başlayabilir ama gerek yok, 1'den başlar. Hoca önce `if`'i döngünün içinde düşündü, sonra `if`'i dışarıya alıp `for`'ları içeride yazdı. İkinci dal, karşılaştırma işareti dışında birincinin aynısıdır:

```
#include <stdio.h>

int DiziMinMaxBul(int [], int, int, int *);

int main() {
    int dizi[5] = {3, 7, 1, 5, 9};
    int indis, sonuc;

    sonuc = DiziMinMaxBul(dizi, 5, 1, &indis);
    printf("%d %d\n", sonuc, indis);
    return 0;
}

int DiziMinMaxBul(int dizi[], int n, int op, int *indis) {
    int i, sonuc = dizi[0];
    *indis = 0;                 /* ders sonrasi eklendi */
    if (op == 1) {
        for (i = 1; i < n; i++)
            if (dizi[i] > sonuc) {
                sonuc = dizi[i];
                *indis = i;
            }
    } else {
        for (i = 1; i < n; i++)
            if (dizi[i] < sonuc) {
                sonuc = dizi[i];
```

```
                *indis = i;
            }
    }
    return sonuc;
}
```

Çalıştırıldığında ekranda `9 4` göründü: En büyük eleman 9, indisi 4.

> **Ders sonrası not:** Altyazıda `*indis` için bir ilk değer verildiği duyulmuyor. Aranan eleman dizinin ilk elemanıysa hiçbir karşılaştırma doğru çıkmaz ve `*indis` atanmamış kalır. Bu yüzden yukarıdaki koda `*indis = 0;` satırı eklendi.

12. Matriste En Büyük ve En Küçük Eleman: Transform and Conquer

Şimdi $m \times n$'lik bir matrisin en büyük veya en küçük elemanını, az önceki fonksiyonu kullanarak bulacağız. Bütün matristen tek bir değer gelecektir. Hocanın düşüncesi şöyledir:

1. Her satırın en büyüğü veya en küçüğü, hazır fonksiyonla bulunabilir.
2. Bu değerler `K` adlı bir dizide tutulur. 50 satır olduğu kabul edilir. Aynı iş sütunlar için de yapılabilirdi.
3. Satırı fonksiyona göndermek için geçici bir `tmp` dizisi kullanılır. Her satır önce `tmp`'ye kopyalanır.
4. `tmp` ile `DiziMinMaxBul` çağrılır, sonuç `K[i]`'ye yazılır.
5. Döngü bitince `K`'nın en büyüğü veya en küçüğü için aynı fonksiyon bir kez daha çağrılır.

Çağrıda eleman sayısı olarak 50 değil `n` verilir, çünkü 50'sinin de dolu olduğu garanti değildir. `K` için ise eleman sayısı `m`'dir, çünkü bu dizi satır sayısına göre dolar. Ayrıca indis değişkeni tanımlanıp adresi gönderilmelidir, fonksiyon imzası bunu ister. Bu sonucu burada kullanmıyoruz, istersek ekrana yazdırabiliriz.

Hoca `K` dizisinin elemanlarının sıfırlanması gerektiğini söyledi ve bunun uzun döngü yerine ilk değer listesiyle yapılabildiğini gösterdi: `int K[50] = {0};` bütün elemanları sıfırlar.

> **Ders sonrası not:** Bu kodda `K`'nın ilk `m` elemanının her biri döngüde atandığı ve yalnız o kadarı kullanıldığı için sıfırlama büyük olasılıkla zorunlu değildir. Yazmakta yine de bir sakınca yoktur.

```
#include <stdio.h>

int DiziMinMaxBul(int [], int, int, int *);
int MatMinMaxBul(int [][50], int, int, int);

int main() {
    int mat[50][50];
    int m, n, i, j;

    scanf("%d %d", &m, &n);
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &mat[i][j]);

    printf("%d\n", MatMinMaxBul(mat, m, n, 1));
    return 0;
}

int DiziMinMaxBul(int dizi[], int n, int op, int *indis) {
    int i, sonuc = dizi[0];
    *indis = 0;
    if (op == 1) {
        for (i = 1; i < n; i++)
            if (dizi[i] > sonuc) {
                sonuc = dizi[i];
                *indis = i;
```

```
            }
    } else {
        for (i = 1; i < n; i++)
            if (dizi[i] < sonuc) {
                sonuc = dizi[i];
                *indis = i;
            }
    }
    return sonuc;
}

int MatMinMaxBul(int mat[][50], int m, int n, int op) {
    int K[50] = {0};
    int tmp[50];
    int i, j, indis, sonuc;

    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++)
            tmp[j] = mat[i][j];
        K[i] = DiziMinMaxBul(tmp, n, op, &indis);
    }
    sonuc = DiziMinMaxBul(K, m, op, &indis);
    return sonuc;
}
```

Matris önce diziye indirgendi, dizi için de aynı fonksiyon yeniden çağrıldı. Hocaya göre bu **transform and conquer** mantığıdır: Var olan bir çözümü başka bir problemin çözümüne uyarlamak. Normalde bu iş iki döngüyle de rahatça yapılabilir. Burada amaç var olan fonksiyonu kullanmaktır.

Program 3×4'lük bir matrisle (12 değer) çalıştırıldı ve en büyük eleman olarak 9 bulundu. Matris ekranda düzenli biçimde yazdırılmadı, doğrudan sonuç gösterildi.

13. En Yakın Nokta Çifti (Closest Pair) Problemine Giriş

Hoca gelecek dersin konusunu kısaca, teorik olarak tanıttı. İki boyutlu bir düzlemde veya n boyutlu bir uzayda noktalarımız olsun. Mesafelerine göre birbirine en yakın iki nokta hangisidir?

Hocaya göre uzaklık temelli yapay zeka sistemlerinin çoğu ve öneri sistemlerinin (recommendation systems) çoğu bu esasa dayanır. Hoca kendi doktora tezinde de bunu kullandığını söyledi.

Hocanın örneği: Öğrencileri iki sayısal özelliğe göre düzleme yerleştirelim: matematik ve tarih sınavı puanları. Her çarpı işareti bir öğrenciyi gösterir. Biri matematikten de tarihten de düşük kalmıştır, başkası matematikte çok iyi, tarihte ortadır. Böyle bin, on bin, milyon örnek olabilir. Uzaklık benzerliği gösterir: En küçük uzaklık, en çok birbirine benzeyen iki öğrenciyi verir. Yani başarı profili en çok benzeyen iki öğrenciyi böyle bulabiliriz.

Hocanın algoritması (bin örnek için):

1. `min` değerini tam sayının en büyük değeri yap.
2. `i` 1'den 1000'e, `j` 1'den 1000'e kadar dön.
3. i. ve j. öğrenci arasındaki uzaklık $d(i, j)$'yi hesapla.
4. $d(i, j) <$ min ise `min` değerini $d(i, j)$ yap, `i` ve `j`'yi sakla.

Böylece bütün olası durumlar hesaplanır.

> **Ders sonrası not:** Tahtadaki taslakta `i` ve `j`'nin ikisi de 1'den başladığı için $i = j$ durumunda uzaklık 0 olur ve en küçük uzaklık hep 0 çıkar. Ayrıca her çift iki kez incelenir. `j`, `i + 1`'den başlatılmalıdır. Böylece 1000 örnek için $1000 \cdot 1000 = 1\,000\,000$ yerine $\frac{1000 \cdot 999}{2} = 499\,500$ uzaklık hesaplanır (python ile doğrulandı). Karmaşıklık yine n^2 düzeyinde kalır.

Boyut sayısı artarsa, örneğin 3 boyutta x, y ve z değerleri olur. Matematik ve tarih yerine 20 çeşit ders olsaydı, uzaklık hesabına yeni boyutlar eklenirdi.

Bu algoritma, n öğrenci için n^2 kez çalışır. Sıralama algoritmalarında olduğu gibi, n^2 büyük değerlerde çok sorunludur. Hoca sordu: Bunu $n \log n$'e, $\log n$'e, n'e veya sabit süreye indirebilir miyiz? $n \log n$ yapılabilmiştir ve bununla ilgili algoritmalar vardır. Bu konu daha çok Algoritma Analizi dersinindir. Bu derste n^2 üzerinde nasıl iyileştirmeler yapılabileceğine bakılacaktır.

> **Ek bilgi (derste anlatılmadı):** Uzaklık olarak büyük olasılıkla Öklid uzaklığı kullanılır. İki boyutta $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ olur, her yeni boyut kök içine bir kare terimi daha ekler. Bütün noktalar en az bir kez okunmak zorunda olduğu için bu problem büyük olasılıkla $\log n$ veya sabit sürede çözülemez.

Dersin Toparlaması

Bu derste bir problemi önce fonksiyonlara bölmeyi, sonra kodlamayı gördük. Kesir işlemlerinde toplama ile çıkarma ve çarpma ile bölme, algoritmik olarak aynı oldukları için tek fonksiyonda birleşti. Fark, çağrıda verilen parametrelerle sağlandı: `-c` ve `d`, `c` yer değişimi. Structure chart, fonksiyonları ve parametrelerini kodlamadan önce görmeyi sağladı. Pointer'ın nerede gerektiğini, `scanf` içindeki `&` ayrımını, `*payda` ile içeriğe bakmayı ve `%c`'den önceki boşluğun nedenini kod üzerinde konuştuk. `refine` çağrısının tek bir yere taşınması ve `gcd`'nin ayrı tutulması, iyi bir tasarımda değişikliğin programın kalanını bozmadığını gösterdi. Stub, low coupling ve high cohesion kavramları ile Öklid algoritması aynı düşünceye bağlandı.

Sonra akış diyagramında fonksiyon gösterimini, parametrik yazmayı ve flag değişkenini gördük. Dizideki en büyük ve en küçük elemanı bulan fonksiyonu matris problemine uyarlayarak transform and conquer mantığını uyguladık. Son olarak gelecek dersin konusu olan en yakın nokta çifti problemine ve n^2 çalışan kaba çözümüne giriş yaptık. Bu çözümdeki iyileştirmeler ve fonksiyonlarla ilgili başka algoritmik örnekler gelecek derste ele alınacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.